Rapport Final de Projet

2éme Année BI

Atelier Génie Logiciel

2025 - 2026

SafeBite

Platforme de commande de plats sans gluten

Réalisé par :

Imen Abed A03

Syrine Hafsie A01

Maram Ayari A01

Elyes Jridi A03

Adem Arfaoui A05

Table de matiére :

1. Fiche Projet 3

2. Product Backlog 4

2.1 User Stories (13 US) 4

3. Sprint Backlogs 6

3.1 Sprint1 planning 6

3.2 Sprint2 planning 8

4. Modelisation UML 9

4.1 Diagramme UC 9

4.2 Diagramme de Classes 10

4.3 Diagramme de Séquence 11

5. Maquettes UI 12

6. Approche Hybride AGL & AI 15

6.1 Prompt AI + Code générer 15

7. Tableau Comparatif AGL vs AI 19

8. Sprint reviews & rétrospectives 20

8.1 Sprint 1 Review + Rétro 20

8.2 Sprint 2 Review + Rétro, ... 21

9. Bilan individuel 21

1. Fiche projet :

Nom du projet : **SafeBite**

Description du projet :

Aujourd'hui, plus de 1 % de la population tunisienne est atteinte de la maladie cœliaque. Ces personnes doivent éviter strictement le gluten, mais aller au restaurant ou commander un repas reste un vrai défi. Le risque de contamination croisée est élevé, et beaucoup de malades vivent dans l'angoisse de tomber malade après un repas.

Problème résolu :

Les personnes atteintes de la maladie cœliaque ont des difficultés à trouver des aliments sans gluten sûrs et à tout moment .

Cible :

- Consommateur qui mangent sans gluten
- Restaurants proposant des plats sans gluten

Acteurs : visiteurs **/** client / restaurant / admin

Fonctionnalités par Membre :

Syrine Hafsia (product backlog + développeuse):

- Création compte client
- Création compte réstaurant
- Login client et réstaurant

Adem Arfaoui (scrum master + développeur):

- Validation des restaurant
- Gestion des utilisateur pour l'admin
- Supprision contenu frudulaux

Maram Ayari (développeuse) :

- Gestion des menu
- Gestion des commandes

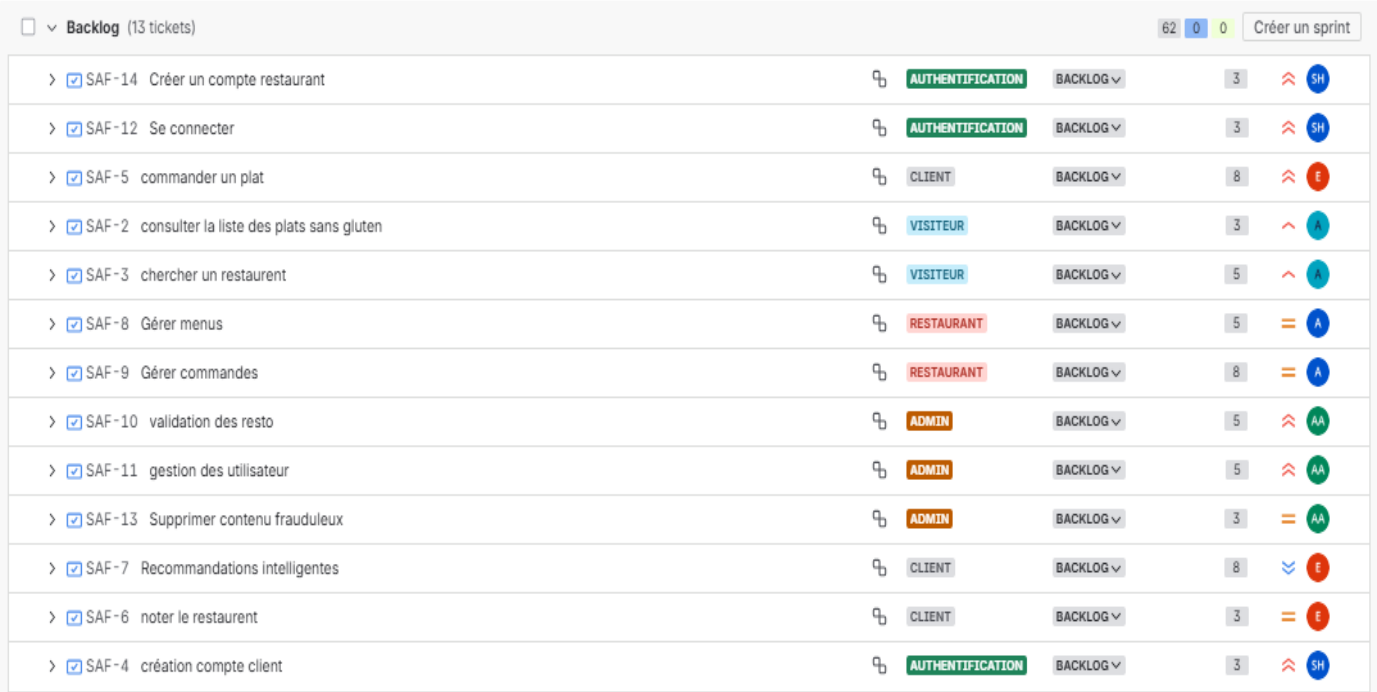
Elyes Jridi (développeur) :

- Commander plat
- Recommandation intelligentes
- Noter restaurant

Imen Abed (développeuse) :

- Consulter liste des plats
- Chercher des restaurants

2. Product Backlog : 13 User Stories



Backlog (13 tickets) — 62 | 0 | 0 — Créer un sprint

SAF-14	Créer un compte restaurant	AUTHENTIFICATION	BACKLOG	3	SH
SAF-12	Se connecter	AUTHENTIFICATION	BACKLOG	3	SH
SAF-5	commander un plat	CLIENT	BACKLOG	8	E
SAF-2	consulter la liste des plats sans gluten	VISITEUR	BACKLOG	3	A
SAF-3	chercher un restaurent	VISITEUR	BACKLOG	5	A
SAF-8	Gérer menus	RESTAURANT	BACKLOG	5	A
SAF-9	Gérer commandes	RESTAURANT	BACKLOG	8	A
SAF-10	validation des resto	ADMIN	BACKLOG	5	AA
SAF-11	gestion des utilisateur	ADMIN	BACKLOG	5	AA
SAF-13	Supprimer contenu frauduleux	ADMIN	BACKLOG	3	AA
SAF-7	Recommandations intelligentes	CLIENT	BACKLOG	8	E
SAF-6	noter le restaurent	CLIENT	BACKLOG	3	E
SAF-4	création compte client	AUTHENTIFICATION	BACKLOG	3	SH

ID	Rôle	User Story (En tant que… je veux… afin de…)	Priorité (MoSCoW)	Critères d'acceptation (≥3)	Story Points	Responsable
US-1	Visiteur	En tant que visiteur, je veux consulter la liste des restaurants afin de découvrir des options sans gluten	Must	- Liste visible sans connexion - Affichage nom + note - Chargement rapide	3	M
US-2	Visiteur	En tant que visiteur, je veux rechercher un restaurant afin de trouver rapidement un lieu spécifique	Must	- Barre de recherche fonctionnelle - Résultats pertinents - Filtrage dynamique	5	M
US-3	Client	En tant que client, je veux créer un compte afin d'accéder aux fonctionnalités avancées	Must	- Formulaire valide - Email unique - Confirmation réussie	3	HS
US-4	Client	En tant que client, je veux me connecter afin d'accéder à mon espace personnel	Must	- Authentification sécurisée - Message d'erreur si incorrect - Redirection après login	3	HS
US-5	Client	En tant que client, je veux consulter les plats sans gluten afin de choisir un repas sûr	Must	- Liste filtrée sans gluten - Détails visibles - Interface claire	5	E
US-6	Client	En tant que client, je veux noter un restaurant afin de partager mon	Should	- Note de 1 à 5 - Un seul vote par utilisateur	3	E

ID	Rôle	User Story (En tant que… je veux… afin de…)	Priorité (MoSCoW)	Critères d'acceptation (≥3)	Story Points	Responsable
		expérience		- Mise à jour immédiate		
US-7	Client	En tant que client, je veux recevoir des recommandations afin de découvrir des restaurants adaptés	Could	- Suggestions personnalisées - Basées sur historique - Affichage clair	8	E
US-8	Restaurant	En tant que restaurant, je veux gérer mon menu afin de proposer mes plats	Must	- Ajouter/modifier/supprimer plat - Sauvegarde correcte - Interface simple	5	A
US-9	Restaurant	En tant que restaurant, je veux gérer les commandes afin de traiter les demandes clients	Must	- Voir commandes reçues - Changer statut - Historique disponible	8	A
US-10	Admin	En tant qu'admin, je veux valider les restaurants afin de garantir la fiabilité	Must	- Liste des demandes - Bouton valider/refuser - Notification envoyée	5	M
US-11	Admin	En tant qu'admin, je veux gérer les utilisateurs afin de maintenir la plateforme	Should	- Voir utilisateurs - Suspendre compte - Modifier infos	5	M
US-12	Admin	En tant qu'admin, je veux supprimer les contenus frauduleux afin de sécuriser la plateforme	Must	- Détection contenu - Suppression rapide - Log des actions	3	W
US-13	Restaurant	En tant que restaurant, je veux créer un compte afin de proposer mes services	Must	- Formulaire complet - Vérification données - Demande envoyée à admin	3	HS

3. Sprint Backlogs :

3.1. Sprint 1 Backlog :

Product Sprint 1 20 avr. – 11 mai (10 tickets) | 0 | 48 | 0 | Terminer le sprint

SAF-4	création compte client	AUTHENTIFICATION	EN COURS	3
SAF-14	Créer un compte restaurant	AUTHENTIFICATION	EN COURS	3
SAF-12	Se connecter	AUTHENTIFICATION	EN COURS	3
SAF-5	commander un plat	CLIENT	EN COURS	8
SAF-2	consulter la liste des plats sans gluten	VISITEUR	EN COURS	3
SAF-3	chercher un restaurent	VISITEUR	EN COURS	5
SAF-8	Gérer menus	RESTAURANT	EN COURS	5
SAF-9	Gérer commandes	RESTAURANT	EN COURS	8
SAF-10	validation des resto	ADMIN	EN COURS	5
SAF-11	gestion des utilisateur	ADMIN	EN COURS	5

Sprint Goal :

Mettre en place les **fonctionnalités de base** : **authentification + navigation + premières interactions client.**

User Stories sélectionnées

- US-3 : Création compte client
- US-13 : Création compte restaurant
- US-4 : Se connecter
- US-5 : Commander un plat
- US-2 : Consulter plats sans gluten
- US-3 : Chercher un restaurant
- US-8 : Gérer menus
- US-9 : Gérer commandes
- US-10 : Validation restaurants
- US-11 : Gestion utilisateurs

Tâches techniques par member :

Syrine Hafsia (Auth) :

- Implémenter Utilisateur.java
- API inscription/login
- Hash password + validation
- Gestion session

Elyes Jridi (Client) :

- UI commande plat
- API commande
- Liaison client → restaurant

Maram Ayari (Restaurant) :

- CRUD menu (Plat.java)
- Interface gestion commandes
- Mise à jour statut commande

Imen Abed (Visiteur)

- Recherche restaurants
- Liste plats sans gluten

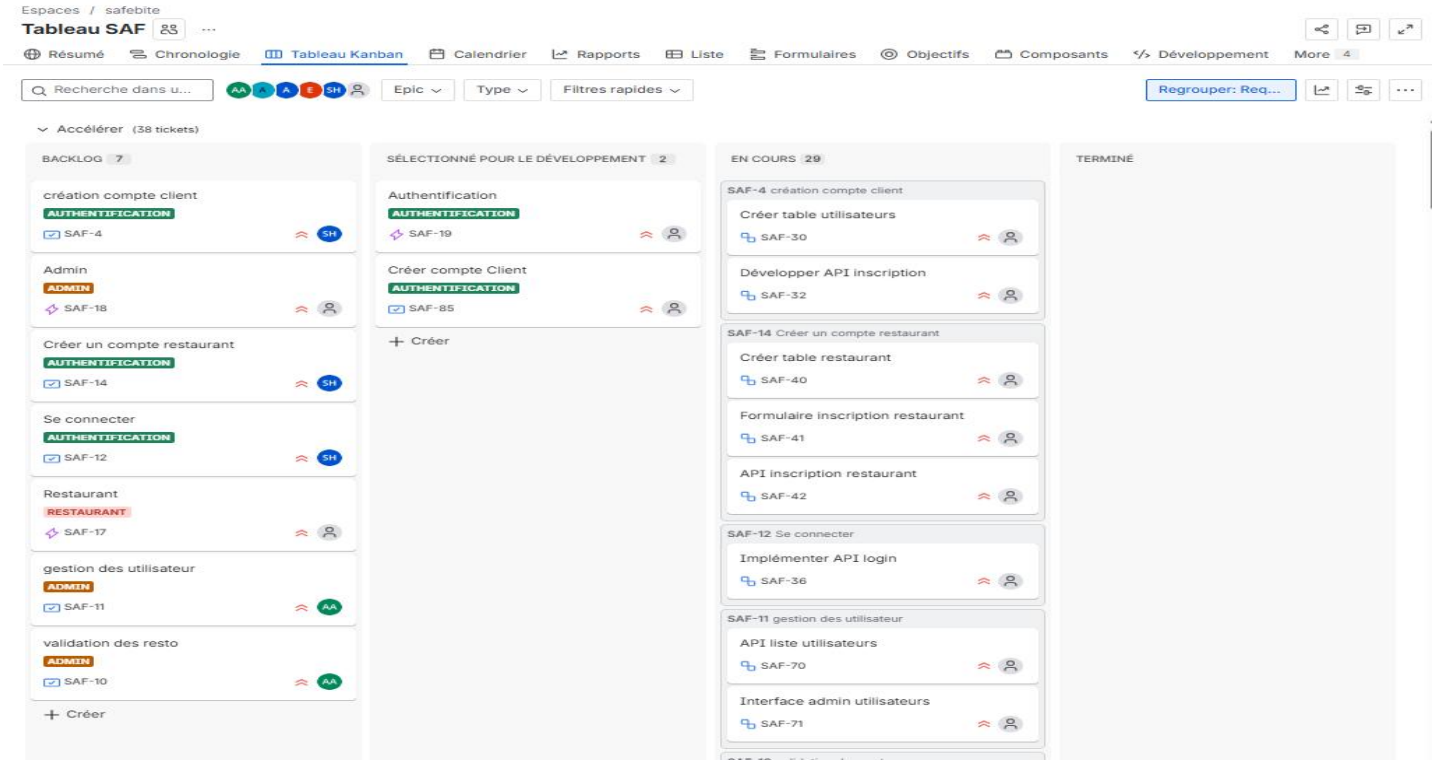
Adem Arfaoui (Admin)

- Validation restaurants
- Gestion utilisateurs

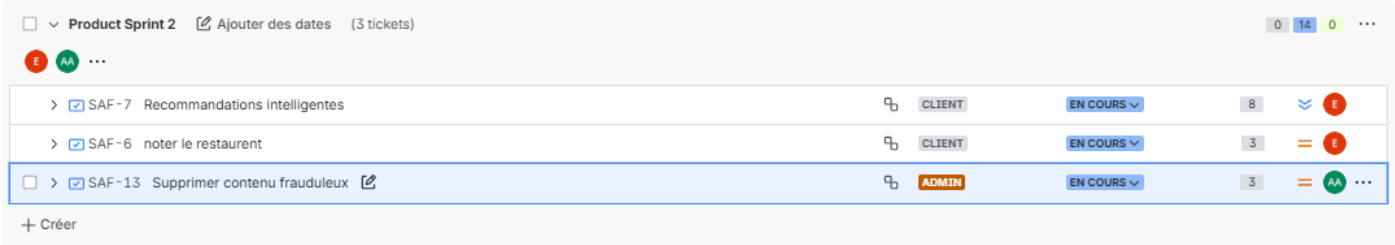
Estimation (Story Points) : Total Sprint 1 ≈ **48 SP**

Definition of Done (DoD) :

- ✓ Code compilable sans erreur
- ✓ Fonction testée (unit test ou manuel)
- ✓ UI fonctionnelle
- ✓ Commit + push GitHub
- ✓ Revue de code validée
- ✓ Intégration réussie



3.2. Sprint 2 Backlog :



Sprint Goal : Améliorer l'expérience utilisateur + ajouter intelligence et feedback

User Stories :

Reportées (si non terminées)

- US-9 : Gérer commandes (si incomplète)
- US-10 : Validation restos

Nouvelles

- US-6 : Noter restaurant
- US-7 : Recommandations intelligentes
- US-12 : Suppression de contenu frauduleux

Ajustement de vélocité :

- Sprint 1 prévu : **48 SP**
- Sprint 1 réalisé (estimé) : **40 SP**
- **Nouvelle vélocité = *40 SP***

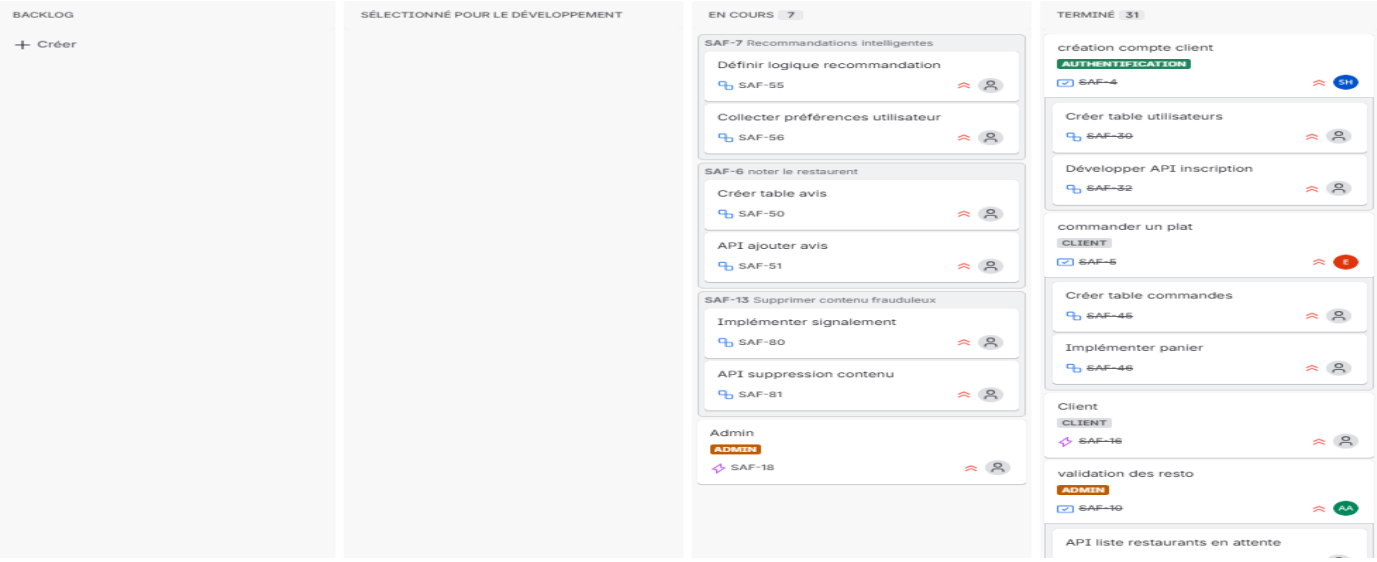
Tâches techniques :

Adem Arfaoui :

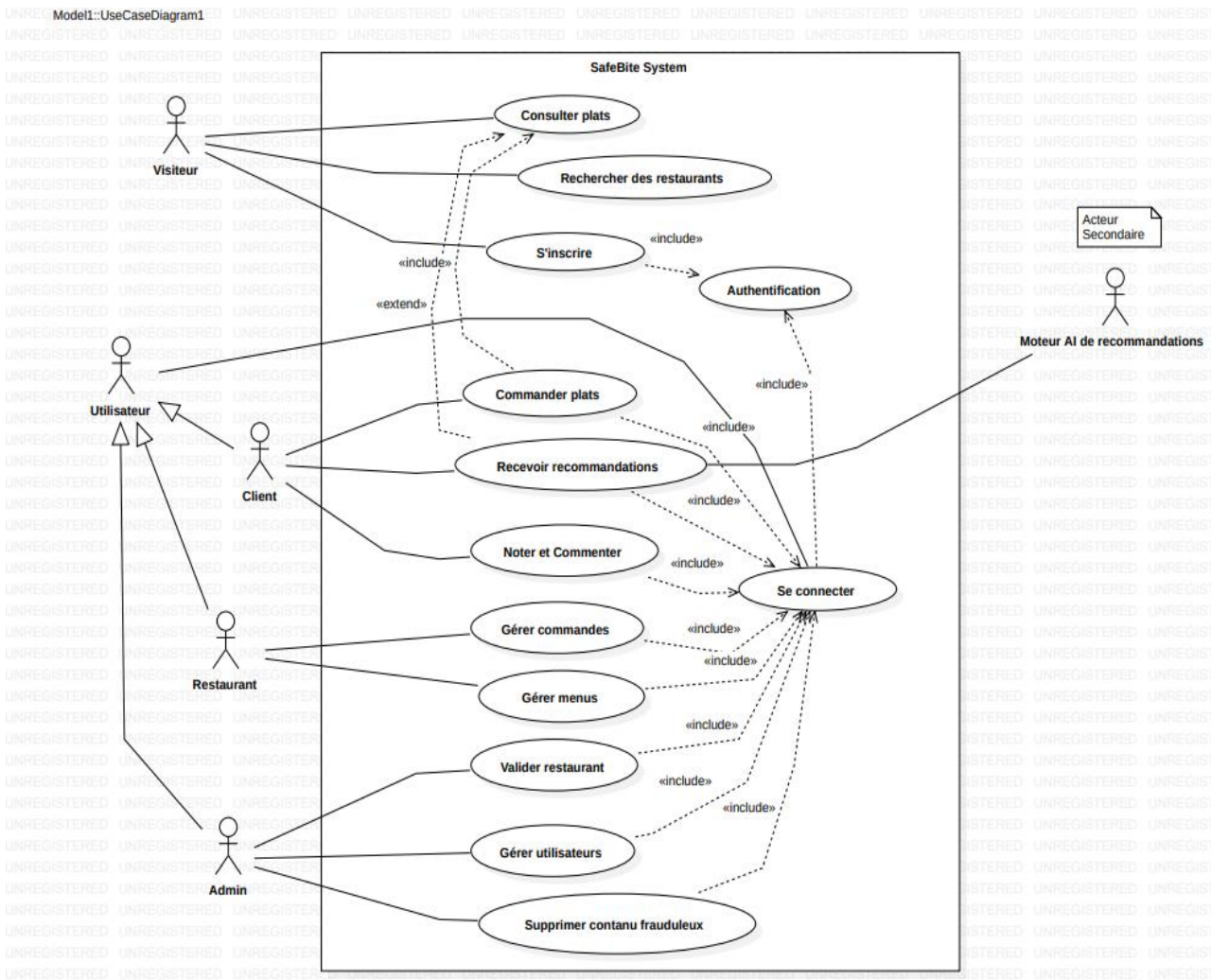
- Suppression de contenu frauduleux

Ilyes Jridi :

- Noter restaurant
- Recommandations intelligentes



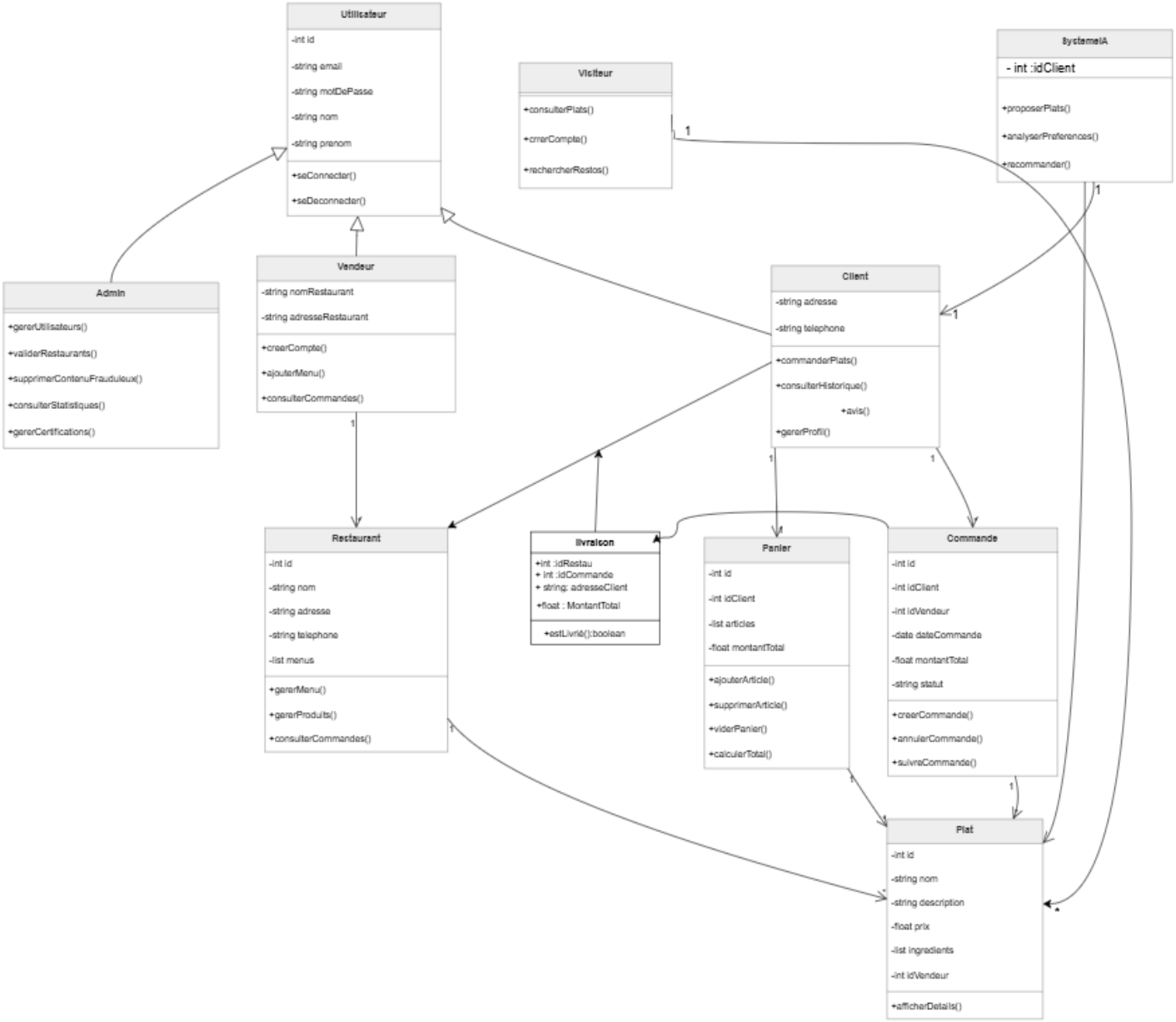
4. Modélisation UML:



4.1. Diagramme de Cas d'Utilisation (Use Case):

Le choix de conception ici repose sur une **hiérarchie d'acteurs** claire pour centraliser les fonctionnalités communes.

- **Spécialisation des Acteurs :** L'acteur Utilisateur est une classe abstraite dont héritent Client, Restaurant et Admin. Cela permet de définir que toute action liée à Se connecter est partagée par tous ces profils via une relation d'inclusion (<<include>>).
- **Séparation Visiteur vs Utilisateur :** Le Visiteur peut consulter les plats et rechercher des restaurants sans compte. L'inscription est une extension optionnelle ou une étape logique menant à l'authentification.
- **Acteur Secondaire (IA) :** L'utilisation du Moteur AI de recommandations comme acteur secondaire montre que le système interagit avec une entité externe pour fournir de la valeur ajoutée (le cas Recevoir recommandations)



4.2. Diagramme de Classes :

Le diagramme de classe privilégie une structure **orientée objet modulaire** pour gérer la complexité des commandes.

- **Héritage (Generalization) :** Comme dans le diagramme de cas d'utilisation, la classe Utilisateur (contenant email, motDePasse, nom) sert de base à Admin, Vendeur (Restaurant) et Client. Cela évite la duplication de données.
- **Gestion des Commandes et du Panier :** Le choix de séparer Panier et Commande permet au client de préparer son achat de manière persistante avant de le valider en une transaction officielle (Commande).
- **Relations Structurelles :** * Un Restaurant possède des Plats. Le Système (ou Moteur AI) est relié au Client pour analyser les préférences et proposer des plats, ce qui traduit techniquement la fonctionnalité de recommandation.

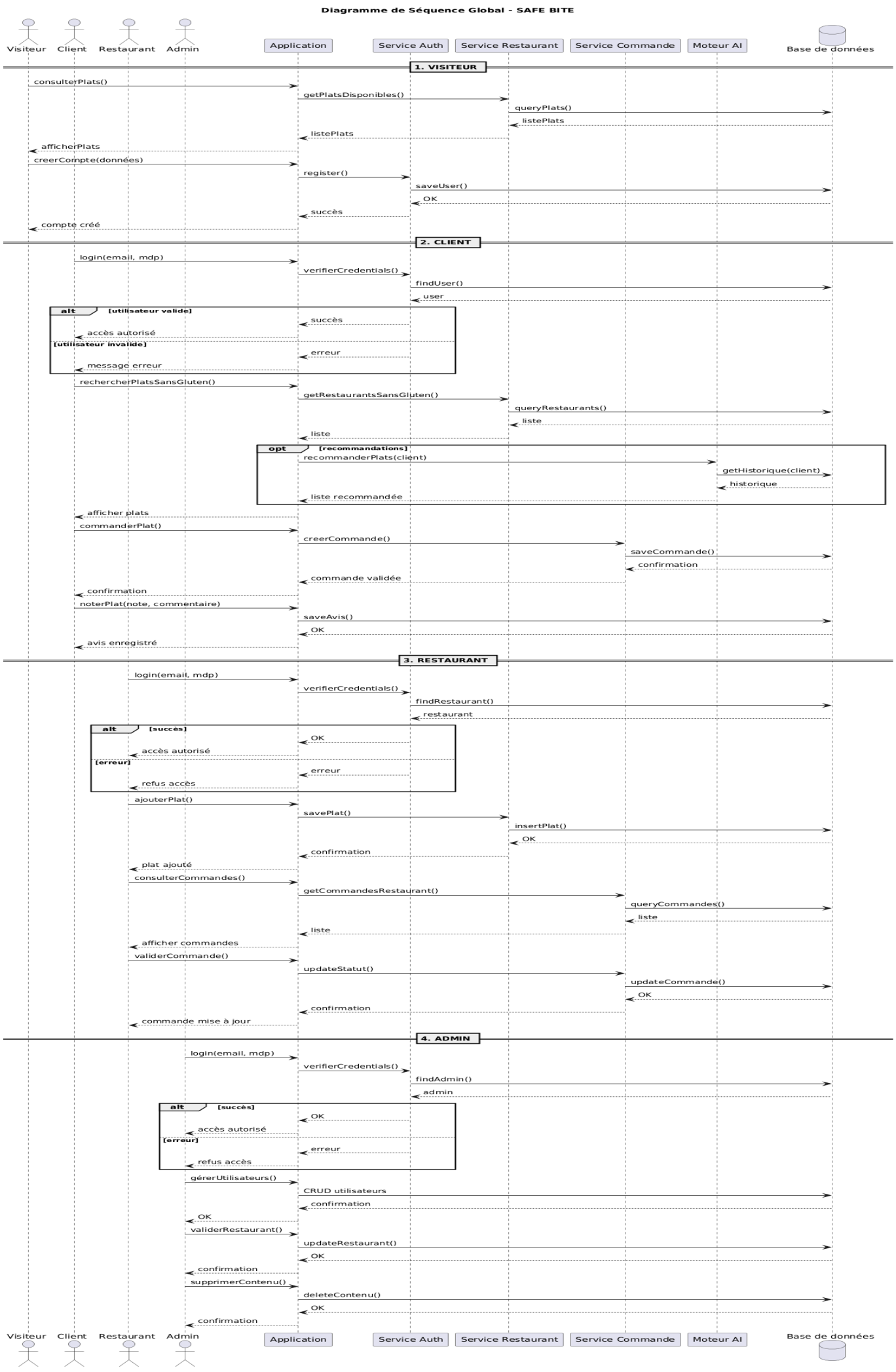


Diagramme de Séquence Global - SAFE BITE
Visiteur
Client
Restaurant
Admin
Application
Service Auth
Service Restaurant
Service Commande
Moteur AI
Base de données
1. VISITEUR
consulterPlats()
getPlatsDisponibles()
queryPlats()
listePlats
listePlats
afficherPlats
creerCompte(données)
register()
saveUser()
OK
succès
compte créé
2. CLIENT
login(email, mdp)
verifierCredentials()
findUser()
user
alt
[utilisateur valide]
succès
accès autorisé
[utilisateur invalide]
erreur
message erreur
rechercherPlatsSansGluten()
getRestaurantsSansGluten()
queryRestaurants()
liste
liste
opt
[recommandations]
recommanderPlats(client)
getHistorique(client)
historique
liste recommandée
afficher plats
commanderPlat()
creerCommande()
saveCommande()
confirmation
commande validée
confirmation
noterPlat(note, commentaire)
saveAvis()
OK
avis enregistré
3. RESTAURANT
login(email, mdp)
verifierCredentials()
findRestaurant()
restaurant
alt
[succès]
OK
accès autorisé
[erreur]
erreur
refus accès
ajouterPlat()
savePlat()
insertPlat()
OK
confirmation
plat ajouté
consulterCommandes()
getCommandesRestaurant()
queryCommandes()
liste
liste
afficher commandes
validerCommande()
updateStatut()
updateCommande()
OK
confirmation
commande mise à jour
4. ADMIN
login(email, mdp)
verifierCredentials()
findAdmin()
admin
alt
[succès]
OK
accès autorisé
[erreur]
erreur
refus accès
gérerUtilisateurs()
CRUD utilisateurs
confirmation
OK
validerRestaurant()
updateRestaurant()
OK
confirmation
supprimerContenu()
deleteContenu()
OK
confirmation

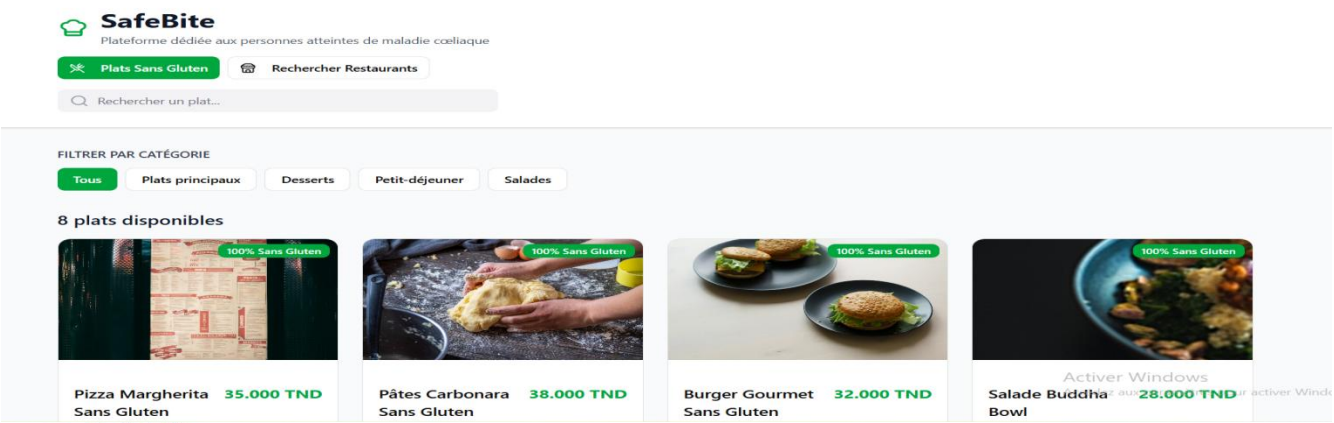
4.3. Diagramme de Séquence :

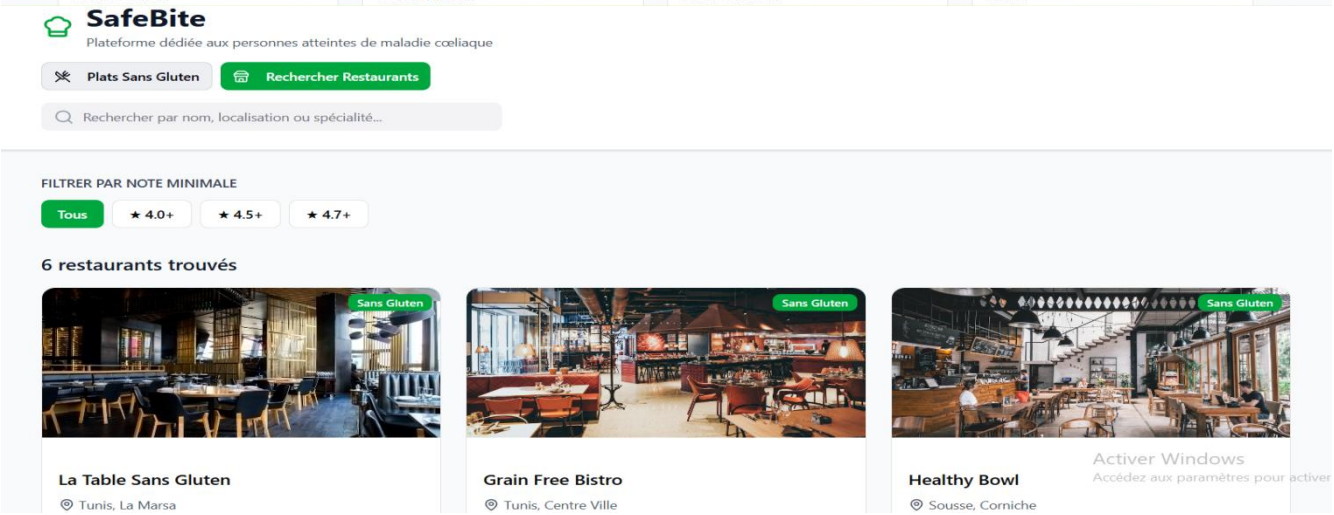
La conception ici suit un modèle de **couches de services** (Architecture 3-tiers ou Microservices).

- **Découpage par Responsabilités :** Au lieu d'avoir un seul bloc logique, le système est divisé en composants spécialisés : Service Auth, Service Restaurant, Service Commande, et Moteur AI. Cela facilite la maintenance et l'évolutivité.
- **Logique de Flux (Fragments Combinés) :**
  - **alt (Alternative) :** Utilisé pour gérer les cas d'erreur (ex: login invalide) ou les conditions de succès, assurant que le système réagit correctement à chaque scénario utilisateur.
  - **opt (Optionnel) :** Utilisé pour l'appel au Moteur AI. Cela montre que l'affichage des plats peut fonctionner même si le moteur de recommandation est indisponible ou non sollicité.
- **Persistance des données :** Chaque action majeure (inscription, ajout de plat, commande) se termine par un appel à la Base de données, garantissant l'intégrité des informations avant de renvoyer un message de confirmation ("OK" ou "succès") à l'utilisateur.

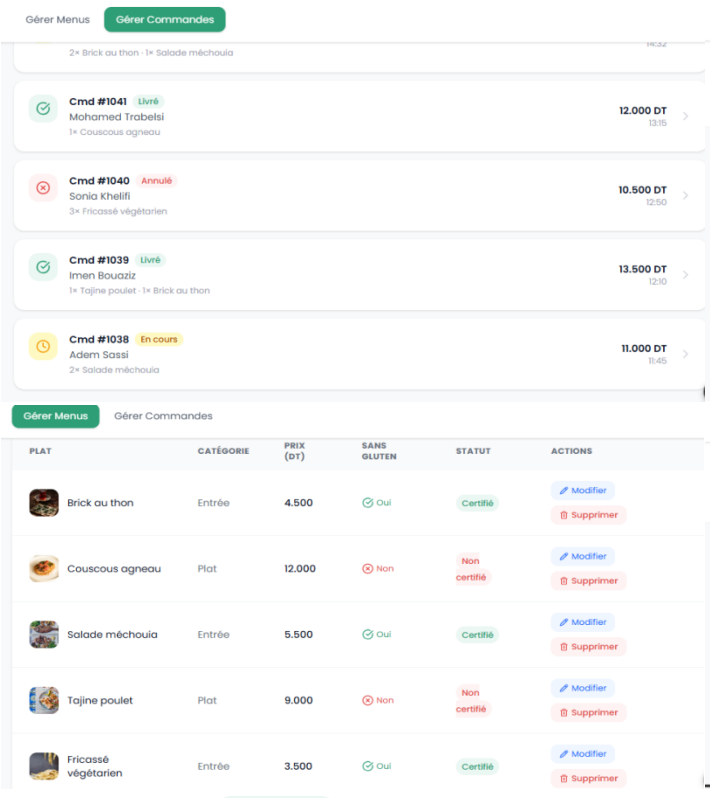
5. Maquettes UI:

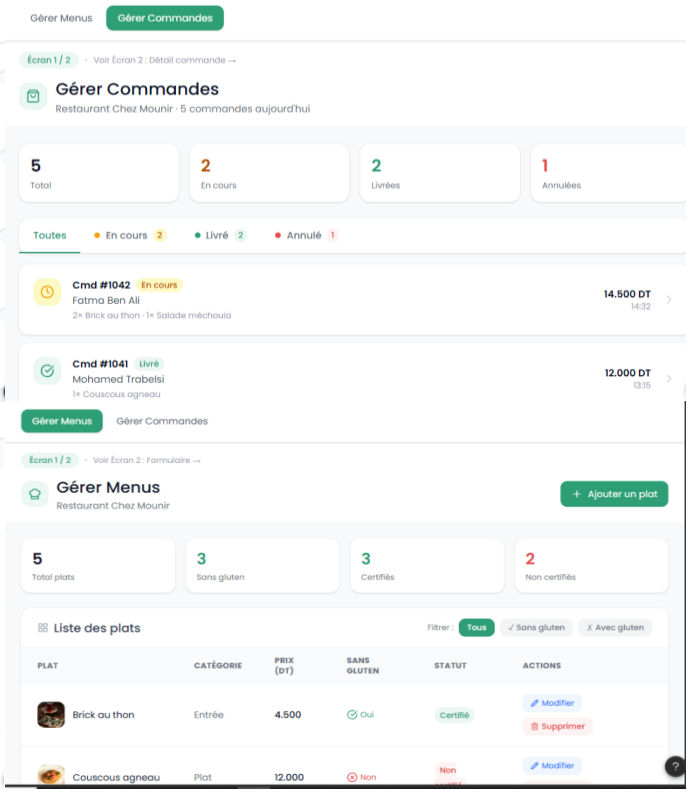
Visiteur :

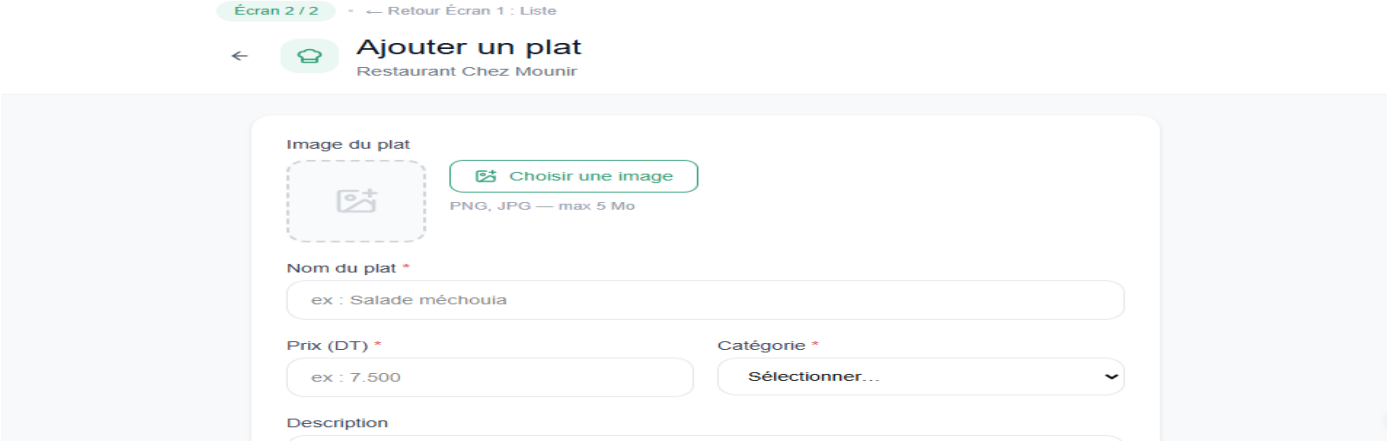




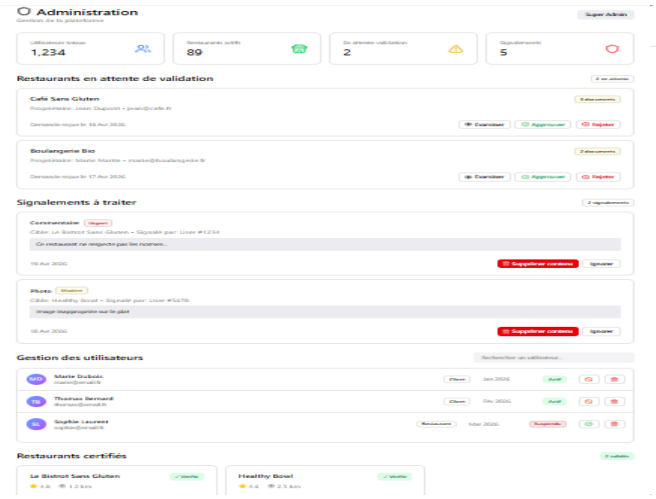
Restaurant:

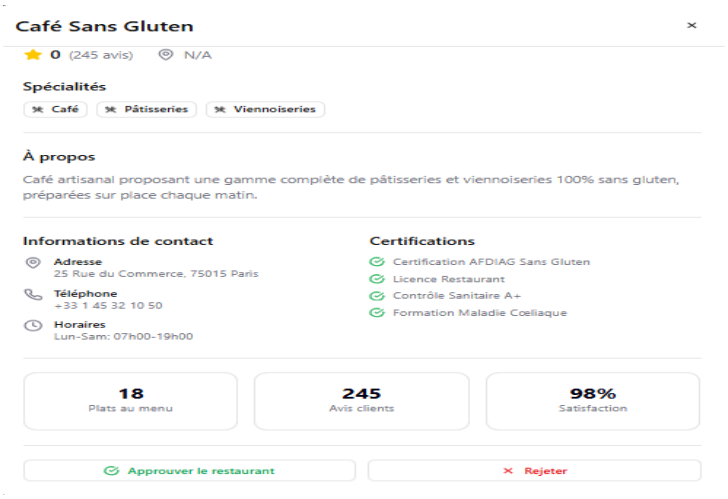




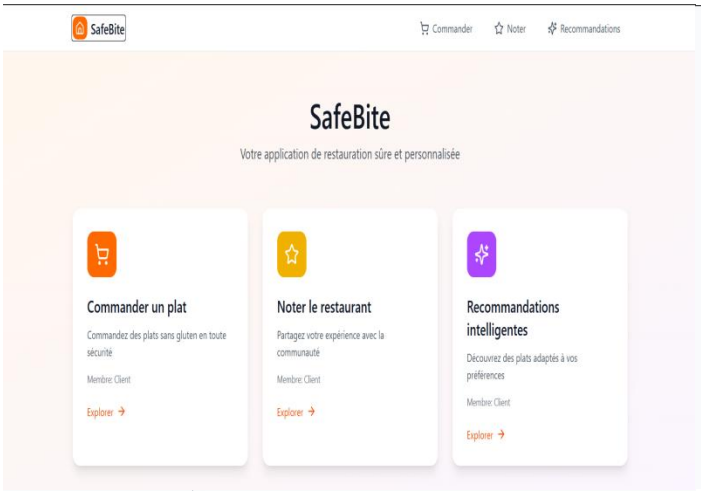


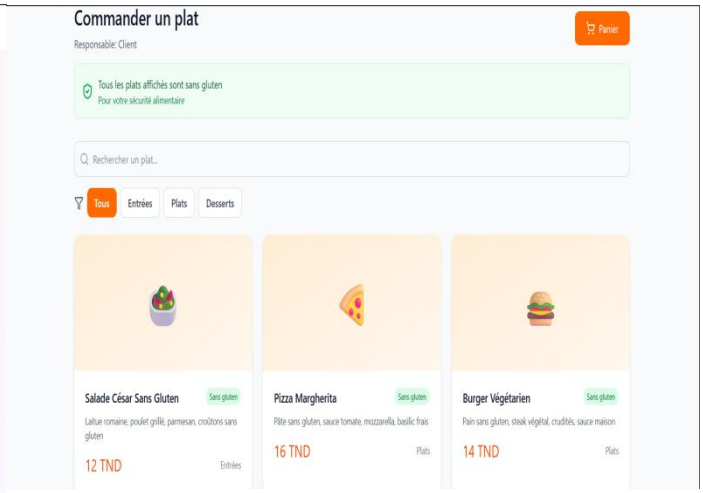
Admin :

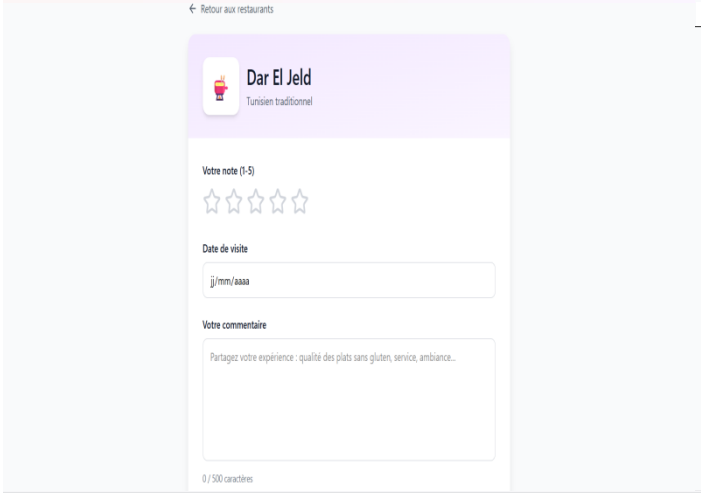


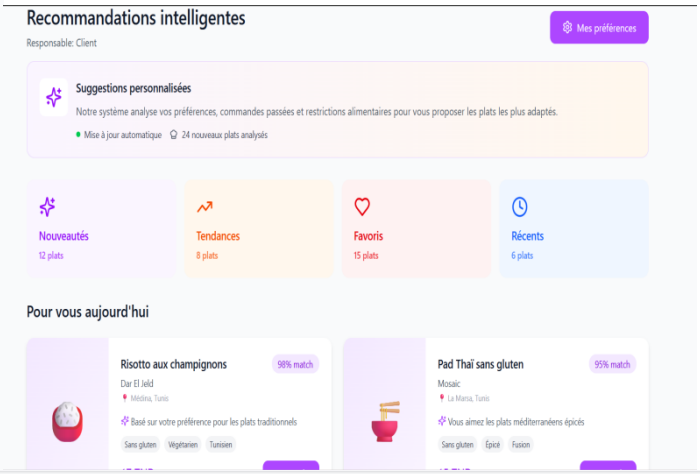


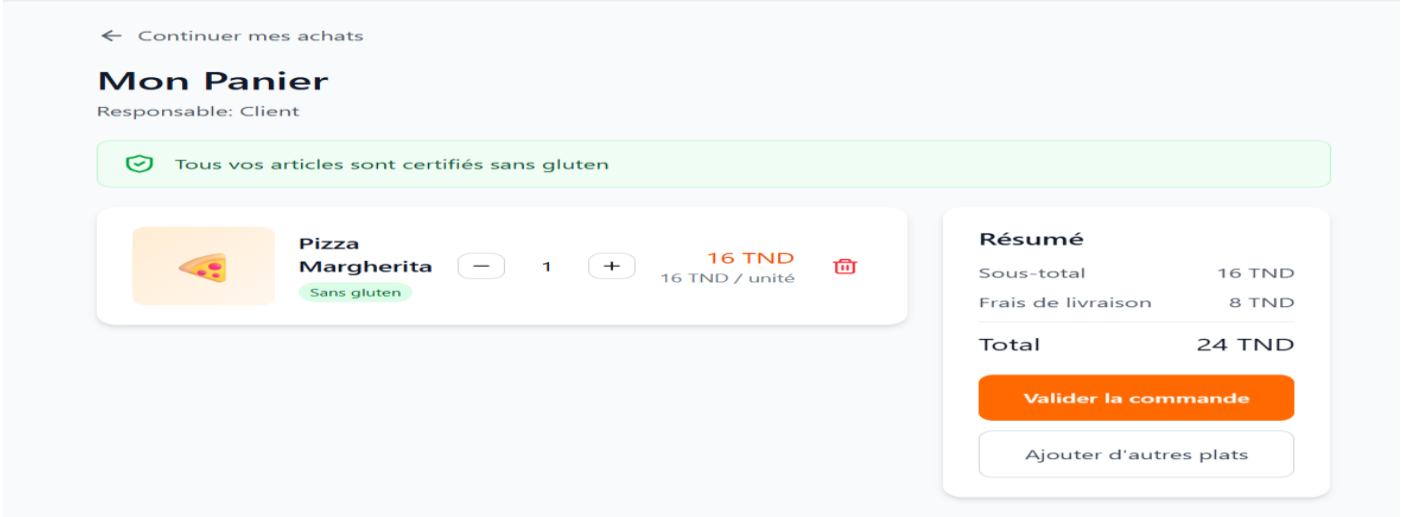
Client :



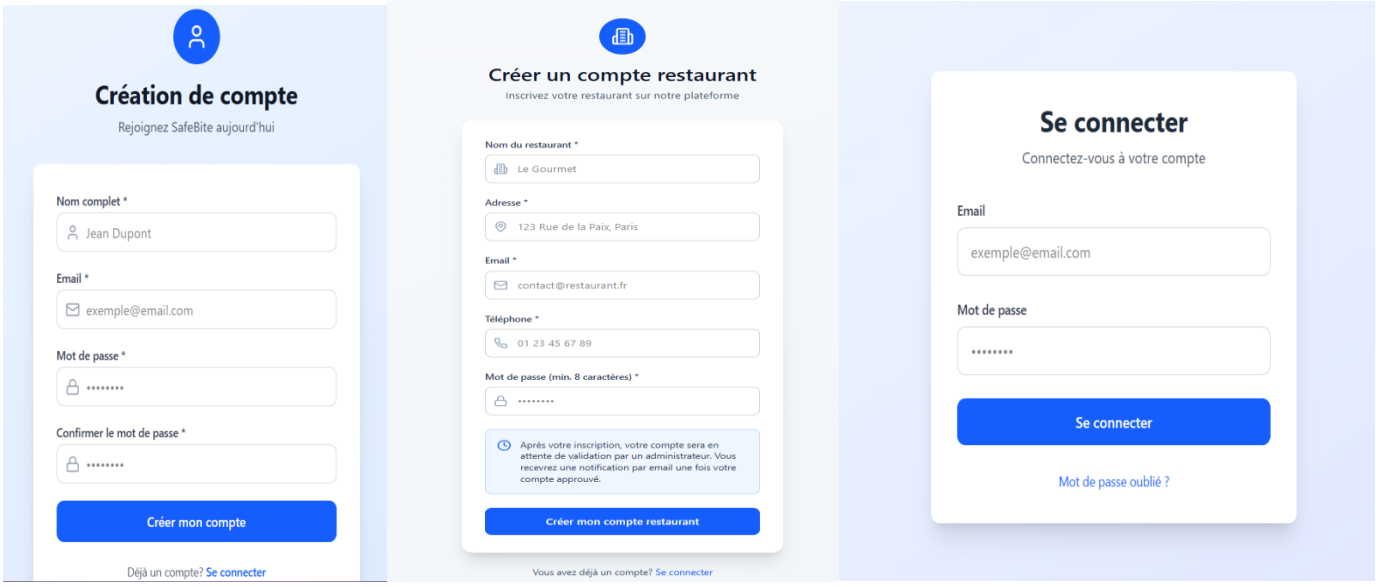








Authentification :



Liste des liens :

https://www.figma.com/make/xwd6k8vwUCOfgICTGzWRLG/SafeBite?t=If4A5h3hV60gOAvE-1 **(Partie Admin)**

- https://www.figma.com/make/EOxM46z7AePXGQq43tyz1z/Cr%C3%A9er-maquettes-pour-fonctionnalit%C3%A9s?fullscreen=1&t=42oblaMxk9Q2gRYV-1&preview-route=%2Forder%2Fmenu **(partie client)**

- https://www.figma.com/make/TS5rttoXQSaVxCaEIdWXZ3/Maquette-d%C3%A9cran?t=6WEt2TLjZINftg8M-1 **(Partie Visiteur)**

- https://www.figma.com/make/6INi9X7eL9QBDsEuwsKKAs/Cr%C3%A9er-maquettes-pour-fonctionnalit%C3%A9s?t=jbpXCWMAFHkYX4le-1&preview-route=%2Ffonctionnalite-1 **(partie restaurant)**

- https://www.figma.com/make/a8j9eFgjFEI5hVdc5Po2ng/login-client?t=DLkWCv63uUsqL4mi-1 **(login client)**

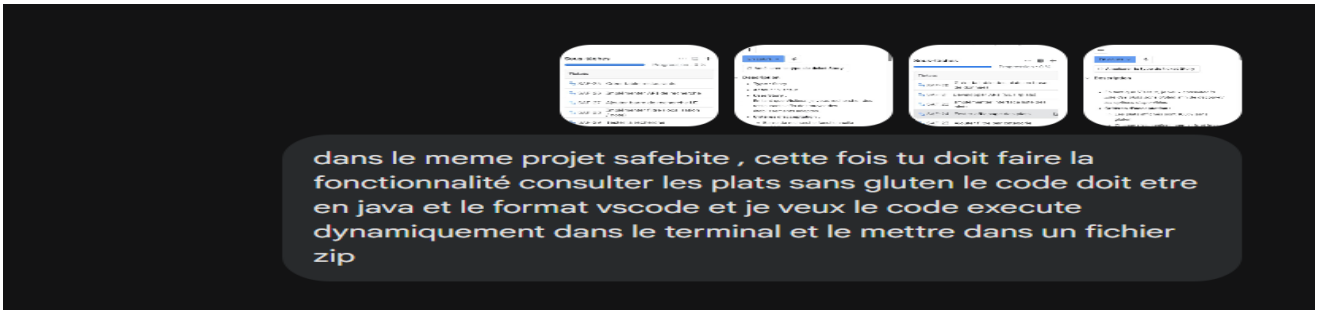
- https://www.figma.com/make/LzjuSwcjofRXvAG7Lfpq8A/login-restaurent?t=POSrF64wLeJJNKpM-1 **(login réstau)**

-https://www.figma.com/make/U5cx9qP8NRwCH9K1m41Utm/cr%C3%A9ation-compte-client?t=9IrFKVCSA3N8Zlz2-1 **(création compte client)**

- https://www.figma.com/make/Kxw14UnNR21L56s7p38SaC/cr%C3%A9ation-compte-r%C3%A9staurent?t=VDGdzvjVbeZhpnuv-1 **(création compte réstau)**

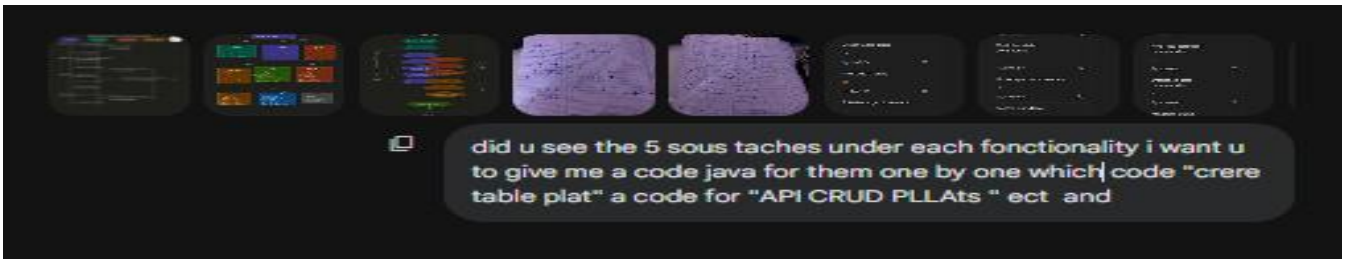
6. Approche Hybride AGL & AI:

Prompt AI : Partie visiteur :

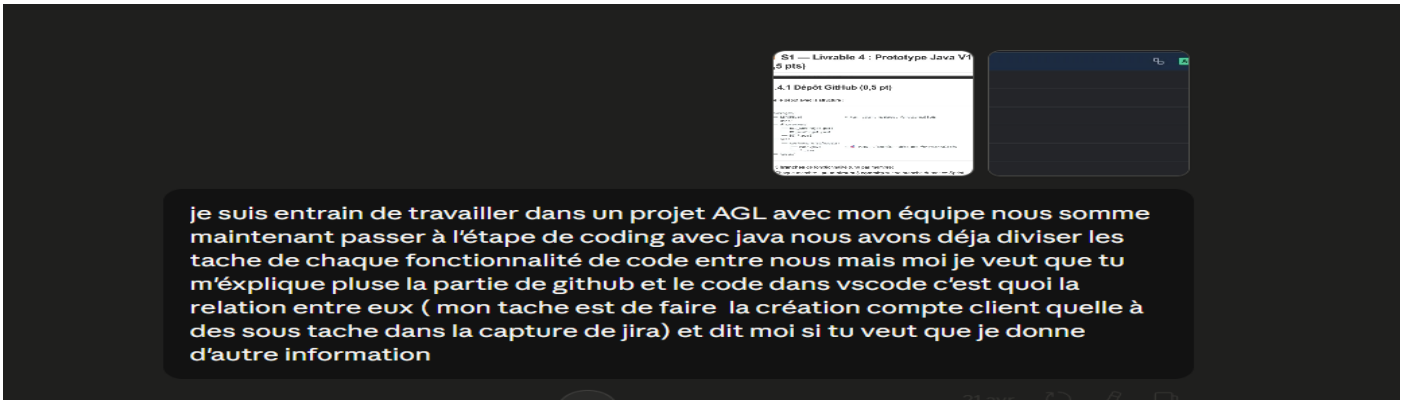


Partie client :

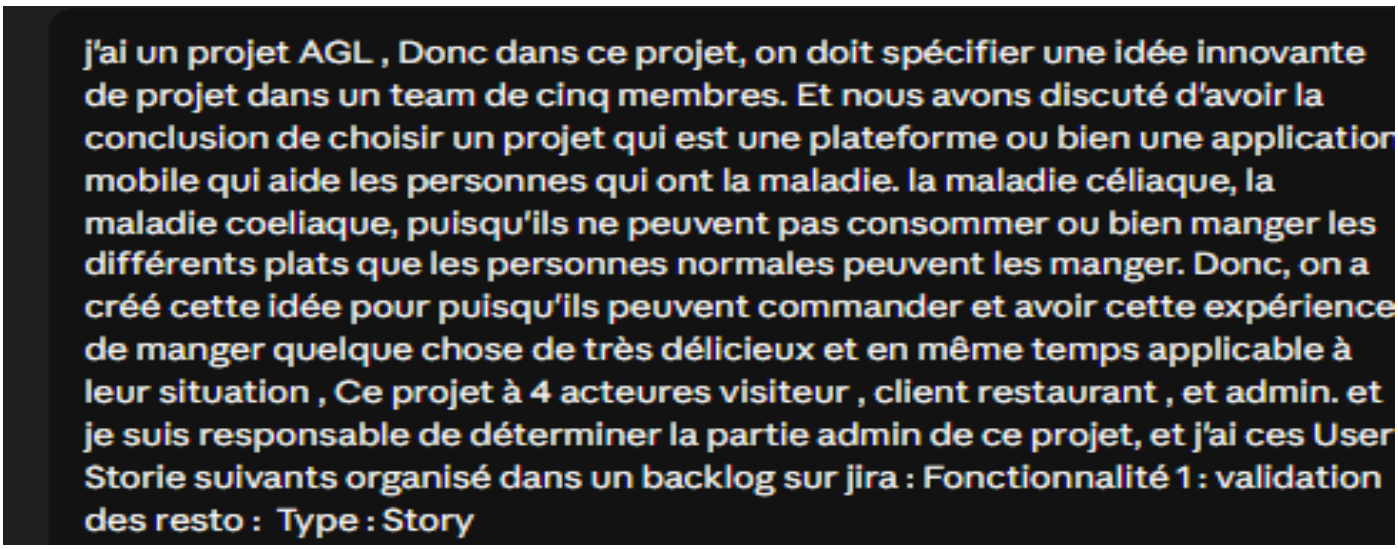
Partie Restaurant :



Partie Auth :



Partie Admin:



Code Générer par le modèle AI : Partie visiteur :

```java
import java.util.Scanner;
import java.util.List;

/**
 * Interface Term
 */
public class main
    Run | Debug
    public static
        Scanner s
        ServiceSa
        boolean c

        System.ou

        while (continuer) {
            System.out.println(x: "\nMenu Principal:");
            System.out.println(x: "1. Consulter la liste des plats");
            System.out.println(x: "2. Chercher un restaurant");
            System.out.println(x: "0. Quitter");
            System.out.print(s: "Action: ");

            String choix = scanner.nextLine();

            switch (choix) {
                case "1":
                    System.out.println(x: "\n--- Liste des Plats ---");
                    service.getTousLesPlats().forEach(System.out::println);
                    break;
```

```
java.util.List

An ordered collection (also known as a sequence). The user of this interface has precise control over where in the list each element is inserted. The user can access elements by their integer index (position in the list), and search for elements in the list.

Unlike sets, lists typically allow duplicate elements. More formally, lists typically allow pairs of elements e1 and e2 such that e1.equals(e2), and they typically allow multiple null elements if they allow null elements at all. It is not inconceivable that someone might wish to implement a list that prohibits duplicates, by throwing runtime exceptions when the user attempts to insert them, but we expect this usage to be rare.

The List interface places additional stipulations, beyond those specified in the Collection interface, on the contracts of the iterator, add, remove, equals, and hashCode methods. Declarations for other inherited methods are also included here for convenience.
```

```java
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

/**
 * Logique métier pour les plats et les restaurants (SAF-21, SAF-23, SAF-26, SAF-28).
 */
public class ServiceSafeBite {
    private List<Plat> cataloguePlats = new ArrayList<>();
    private List<Restaurant> listeRestaurants = new ArrayList<>();

    public ServiceSafeBite() {
        // Données fictives Plats
        cataloguePlats.add(new Plat(nom: "Couscous", prix: 15.0, imageRef: "couscous.jpg", categorie: "Tunisien"));
        cataloguePlats.add(new Plat(nom: "Pizza", prix: 12.5, imageRef: "pizza.jpg", categorie: "Italien"));
        cataloguePlats.add(new Plat(nom: "Salade", prix: 8.0, imageRef: "salade.jpg", categorie: "Healthy"));

        // Données fictives Restaurants
        listeRestaurants.add(new Restaurant(nom: "Le SafSaf", localisation: "La Marsa", note: 4.5));
        listeRestaurants.add(new Restaurant(nom: "La Tavola", localisation: "Tunis", note: 4.2));
        listeRestaurants.add(new Restaurant(nom: "Fast Food Express", localisation: "Sousse", note: 3.5));
    }

    // --- Gestion des Plats ---
    public List<Plat> filtrerPlats(String categorie) {
        return cataloguePlats.stream()
                .filter(p -> p.getCategorie().equalsIgnoreCase(categorie))
                .collect(Collectors.toList());
    }
```

```java
/**
 Modèle pour un restaurant (SAF-25).
 */
public class Restaurant {
    private String nom;
    private String localisation;
    private double note;

    public Restaurant(String nom, String localisation, double note) {
        this.nom = nom;
        this.localisation = localisation;
        this.note = note;
    }

    public String getNom() { return nom; }
    public String getLocalisation() { return localisation; }
    public double getNote() { return note; }

    @Override
    public String toString() {
        return String.format(format: "Restaurant: %s | Ville: %s | Note: %.1f/5", nom, localisation, note);
    }
}
```

```java
/**
 * Modèle pour un plat (SAF-20).
 */
public class Plat {
    private String nom;
    private double prix;
    private String imageRef;
    private String categorie;

    public Plat(String nom, double prix, String imageRef, String categorie) {
        this.nom = nom;
        this.prix = prix;
        this.imageRef = imageRef;
        this.categorie = categorie;
    }

    public String getCategorie() { return categorie; }

    @Override
    public String toString() {
        return String.format(format: "[%s] %s - %.2f DT (Image: %s)", categorie, nom, prix, imageRef);
    }
}
```

Partie client :

```java
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

/**
 * Logique de recommandation intelligente (SAF-55 et SAF-57).
 */
public class IA_Recommandation {
    private List<Plat> cataloguePlats;

    public IA_Recommandation() {
        cataloguePlats = new ArrayList<>();
        // Simulation d'une base de données de plats
        cataloguePlats.add(new Plat(nom: "Salade César", categorie: "Healthy", prix: 15.5));
        cataloguePlats.add(new Plat(nom: "Burger Gourmet", categorie: "Fast-food", prix: 22.0));
        cataloguePlats.add(new Plat(nom: "Pizza Margherita", categorie: "Italien", prix: 18.0));
        cataloguePlats.add(new Plat(nom: "Bol de Quinoa", categorie: "Healthy", prix: 14.0));
        cataloguePlats.add(new Plat(nom: "Pâtes Carbonara", categorie: "Italien", prix: 20.0));
    }

    /**
     * Analyse les préférences et retourne des suggestions filtrées (Critères d'acceptation).
     */
    public List<Plat> genererSuggestions(String preference) {
        return cataloguePlats.stream()
            .filter(p -> p.getCategorie().equalsIgnoreCase(preference))
            .collect(Collectors.toList());
    }
}
```

```java
/**
 * Représente la table des avis (SAF-50).
 */
public class Avis {
    private String client;
    private int note;
    private String commentaire;

    public Avis(String client, int note, String commentaire) {
        this.client = client;
        this.note = note;
        this.commentaire = commentaire;
    }

    @Override
    public String toString() {
        String etoiles = "*".repeat(note) + "-".repeat(5 - note);
        return String.format(format: "[%s] %s : %s", etoiles, client, commentaire);
    }
}
```

```java
import java.util.ArrayList;
import java.util.List;

/**
 * Logique de gestion des avis (SAF-51 et SAF-53).
 */
public class GestionnaireAvis {
    private List<Avis> baseDeDonneesAvis = new ArrayList<>();

    public void ajouterAvis(Avis avis) {
        baseDeDonneesAvis.add(avis);
        System.out.println(x: "> Avis enregistré avec succès.");
    }

    public void afficherAvisPublics() {
        if (baseDeDonneesAvis.isEmpty()) {
            System.out.println(x: "> Aucun avis disponible pour ce restaurant.");
            return;
        }
        System.out.println(x: "\n--- AVIS PUBLICS ---");
        for (Avis avis : baseDeDonneesAvis) {
            System.out.println(avis.toString());
        }
        System.out.println(x: "-------------------\n");
    }
}
```

Partie Restaurant :

Java

```java
package com.safebite.utils;
import java.util.Scanner;

public class InputScanner {
    private static final Scanner scanner = new Scanner(System.in);

    public static String readString() { return scanner.next(); }

    public static int readInt() {
        while (!scanner.hasNextInt()) {
            System.out.print("Veuillez entrer un nombre entier : ");
            scanner.next();
        }
        return scanner.nextInt();
    }

    public static double readDouble() {
        while (!scanner.hasNextDouble()) {
            System.out.print("Veuillez entrer un prix valide (ex: 12,5) : ");
            scanner.next();
        }
        return scanner.nextDouble();
    }
}
```

Java

```java
package com.safebite.models;
import java.util.List;

public class Commande {
    private int id;
    private List<Plat> plats;
    private String statut; // "En cours", "Livré"

    public Commande(int id, List<Plat> plats) {
        this.id = id;
        this.plats = plats;
        this.statut = "En cours";
    }

    public int getId() { return id; }
    public String getStatut() { return statut; }
    public void setStatut(String statut) { this.statut = statut; }
    public List<Plat> getPlats() { return plats; }

    @Override
    public String toString() {
        return "Commande #" + id + " [" + statut + "] - " + plats.size() + " plat
    }
}
```

Java

```java
package com.safebite.models;

public class Plat {
    private int id;
    private String nom;
    private double prix;
    private String imagePath;

    public Plat(int id, String nom, double prix, String imagePath) {
        this.id = id;
        this.nom = nom;
        this.prix = prix;
        this.imagePath = imagePath;
    }

    public int getId() { return id; }
    public String getNom() { return nom; }
    public void setNom(String nom) { this.nom = nom; }
    public double getPrix() { return prix; }
    public void setPrix(double prix) { this.prix = prix; }
    public String getImagePath() { return imagePath; }

    @Override
    public String toString() {
        return String.format("[%d] %-15s | %.2f DT | Image: %s", id, nom, prix, 
    }
}
```

Partie Auth :

```java
package model;

import java.util.ArrayList;
import java.util.List;

/**
 * Représente un client de la plateforme SafeBite.
 * Peut consulter des plats, passer des commandes et laisser des avis.
 */
public class Client extends Utilisateur {

    private List<Commande> historiqueCommandes;
    private List<String> preferences; // ex: "Healthy", "Italien"

    public Client(String nom, String prenom, String email, String motDePasse) {
        super(nom, prenom, email, motDePasse);
        this.historiqueCommandes = new ArrayList<>();
        this.preferences = new ArrayList<>();
    }

    @Override
    public String getRole() {
        return "CLIENT";
    }

    public void ajouterCommande(Commande commande) {
        historiqueCommandes.add(commande);
    }

    public List<Commande> getHistoriqueCommandes() {
```

```java
package model;

import java.util.Scanner;
import service.AdminService;
import service.CompteService;

public class FormulaireCompte {

    private final CompteService service;
    private final AdminService adminService;
    private final Scanner scanner;

    public FormulaireCompte(CompteService service, AdminService adminService) {
        this.service = service;
        this.adminService = adminService;
        this.scanner = new Scanner(System.in);
    }

    public void afficherFormulaire() {
        System.out.println("\n===== 📝 CRÉATION DE COMPTE CLIENT =====");
        System.out.print("Nom       : ");
        String nom = scanner.nextLine();

        System.out.print("Prénom    : ");
        String prenom = scanner.nextLine();
```

```java
package model;      You, 3 hours ago • ajoutt des fonctionnalites de partie Auther

import java.util.ArrayList;
import java.util.List;

You, 3 hours ago | 1 author (You)
/**
 * Représente un client de la plateforme SafeBite.
 * Peut consulter des plats, passer des commandes et laisser des avis.
 */
public class Client extends Utilisateur {

    private List<Commande> historiqueCommandes;
    private List<String> preferences; // ex: "Healthy", "Italien"

    public Client(String nom, String prenom, String email, String motDePasse) {
        super(nom, prenom, email, motDePasse);
        this.historiqueCommandes = new ArrayList<>();
        this.preferences = new ArrayList<>();
    }

    @Override
    public String getRole() {
        return "CLIENT";
    }

    public void ajouterCommande(Commande commande) {
        historiqueCommandes.add(commande);
    }
```

```java
package model;

import java.util.Scanner;
import service.AdminService;
import service.CompteService;

You, 3 hours ago | 1 author (You)
public class FormulaireCompte {

    private final CompteService service;
    private final AdminService adminService;
    private final Scanner scanner;

    public FormulaireCompte(CompteService service, AdminService adminService) {
        this.service = service;
        this.adminService = adminService;
        this.scanner = new Scanner(System.in);
    }

    public void afficherFormulaire() {
        System.out.println("\n===== 📝 CRÉATION DE COMPTE CLIENT =====");
        System.out.print("Nom       : ");
        String nom = scanner.nextLine();
```

Partie Admin :

```java
public class Utilisateur {

    private int    id;
    private String nom;
    private String prenom;
    private String email;
    private String role;          // CLIENT, RESTAURANT, ADMIN
    private UserStatus statut;
    private String dateInscription;

    public Utilisateur() {}

    public Utilisateur(int id, String nom, String prenom, String email,
                       String role, UserStatus statut, String dateInscription) {
        this.id              = id;
        this.nom             = nom;
        this.prenom          = prenom;
        this.email           = email;
        this.role            = role;
        this.statut          = statut;
        this.dateInscription = dateInscription;
    }
```

```java
package com.celiac.admin.dao;

import com.celiac.admin.enums.RestaurantStatus;
import com.celiac.admin.model.Restaurant;
import com.celiac.admin.util.DatabaseConnection;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

/**
 * DAO Restaurant - Accès base de données via JDBC pur (pas de JPA / Hibernate)
 *
 * Sous-tâches couvertes :
 *   ST-1 : API liste restaurants en attente
 *   ST-3 : Boutons accepter / refuser  →  updateStatut()
 */
public class RestaurantDAO {

    // Overridable dans les tests pour injecter une connexion H2 en mémoire
    protected Connection getConn() throws SQLException {
        return getConn();
    }
```

Pour voir le code final , naviguer ver ce lien :

https://github.com/Adem-ARFAOUI/Projet-AGL-2-BI.git

Justification des modifications :

Il s'agit de plusieurs de modifications entre le code générer et la version finale d'une façon que je ne peut pas les mettres tous dans ce document , donc j'ai proposé un exemple :

Code généré par l'IA :

```
public void ajouterAvis(String avis) {
    System.out.println("Avis ajouté : " + avis);
}
```

Problème :

- Le code affiche seulement l'avis mais ne le stocke pas
- Aucune structure de données pour sauvegarder les avis
- Impossible de réutiliser les avis après

Code final corrigé :

```
private List<String> avisList = new ArrayList<>();

public void ajouterAvis(String avis) {
    avisList.add(avis);
    System.out.println("Avis ajouté avec succès");
}
```

Justification des modifications :

Le code généré par l'IA ne respectait pas la logique métier attendue, car il se limitait à afficher l'avis sans le sauvegarder.
Nous avons modifié la méthode pour intégrer une liste (avisList) permettant de stocker les avis des utilisateurs.
Cette correction permet de rendre la fonctionnalité persistante et exploitable dans d'autres parties du programme (affichage des avis, calcul de moyenne, etc.).

7. Tableau comparative AGL vs IA :

Critère	AGL	IA (Intelligence Artificielle)
Objectif	Développer un logiciel structuré, fiable et maintenable	Générer du contenu (code, texte, idées) rapidement
Méthode de travail	Processus organisé (Scrum, UML, cycles de vie)	Basée sur des prompts et des réponses automatiques
Planification	Très importante (backlog, sprints, estimation)	Peu ou pas de planification
Rôle humain	Essentiel (analyse, conception, validation)	Assistance au développeur
Vitesse de développement	Moyenne (dépend de l'équipe)	Très rapide (génération instantanée)

Critère	AGL	IA (Intelligence Artificielle)
Qualité du code	Contrôlée, testée et optimisée	Variable (peut contenir des erreurs)
Compréhension métier	Forte (analyse du besoin)	Limitée (basée sur le prompt)
Maintenance	Facile grâce à la structure du code	Difficile si le code n'est pas bien compris
Tests et validation	Intégrés dans le processus	Nécessite validation humaine
Adaptabilité	Bonne (grâce aux méthodes agiles)	Bonne mais dépend des instructions données
Documentation	Obligatoire et structurée	Souvent absente ou générée automatiquement
Risques	Délais, mauvaise estimation	Code incorrect, sécurité, incohérences

8. Sprint Reviews & Rétrospectives:

8.1. Sprint 1 — Review :

Objectif : Créer les maquettes UI et un prototype Java V1 fonctionnel.

Livrables :

Maquettes UI :

- Parcours : Accueil → Recherche → Résultats → Détails
- Écrans : Inscription, Recherche, Consultation, Avis, Admin

Prototype V1 :

- Main.java fonctionnel (création utilisateur, recherche, ajout restaurant et avis)
- Interaction simple entre classes (Utilisateur, Restaurant, Avis, Admin)

User Stories :

Done : inscription, recherche, affichage, ajout restaurant et avis

Reportées : filtres avancés, notation, validation admin, gestion profils

Feedback PO : Projet clair et utile, UI correcte Améliorations : géolocalisation, meilleure UI, plus de détails

Rétrospective :

Positifs : organisation, GitHub, maquettes utiles

À améliorer : estimation, communication, intégration, charge

Actions : daily meetings, meilleure planification, tests plus tôt

8.2. Sprint 2 — Review :

Objectif : Prototype complet avec logique métier avancée.

Livrables :

Prototype V2 :

- Scénarios complets (compte, recherche, avis, notation)
- Logique métier enrichie

Fonctionnalités :

- Recherche avec filtres
- Notation moyenne
- Gestion utilisateurs + admin

User Stories

Done : toutes les principales US

Partiel : UI console, validations à améliorer

Feedback PO : Très satisfait, système fonctionnel Suggestions : version web, carte, favoris.

Rétrospective :

Positifs : meilleure orga, communication, intégration, usage IA

À améliorer : code complexe, tests, documentation

Actions : tests unitaires, refactoring, version web, optimisation

9. Bilan individuel:

Adem Arfaoui — Développement partie Admin

Dans ce projet, j'ai été responsable du développement de la partie Admin.
J'ai travaillé sur la gestion des utilisateurs, la validation des restaurants et le contrôle global de la plateforme.
J'ai appris à mieux structurer une application en Java en utilisant les principes de la programmation orientée objet.
J'ai également amélioré mes compétences en logique métier et en organisation du code.
Une des principales difficultés rencontrées était la coordination avec les autres modules du projet pour assurer une bonne intégration.
Cette expérience m'a permis de mieux comprendre le rôle critique de l'administration dans une application web.

Syrine Hafsia — Développement partie Authentification

J’ai travaillé sur la partie authentification du projet, incluant l’inscription et la connexion des utilisateurs.
J’ai appris à gérer la validation des données utilisateurs et à sécuriser les accès au système.
Ce travail m’a permis de comprendre l’importance de la sécurité dans une application web.
J’ai rencontré des difficultés dans la gestion des cas d’erreurs et la validation des champs (email, mot de passe).
Grâce à ce projet, j’ai amélioré mes compétences en Java et en gestion des formulaires utilisateur.

Maram Ayari — Développement partie Restaurant

J’ai été responsable de la gestion des restaurants dans l’application.
J’ai développé les fonctionnalités d’ajout, de modification et d’affichage des restaurants sans gluten.
J’ai appris à manipuler les objets en Java et à organiser les données de manière structurée.
Une difficulté rencontrée était la modélisation correcte des relations entre les restaurants et les autres classes.
Ce projet m’a aidée à comprendre l’importance de la qualité des données dans une application réelle.

Imen Abed — Développement partie Visiteur

J’ai travaillé sur la partie visiteur de l’application, qui permet de consulter les restaurants sans créer de compte.
J’ai développé les fonctionnalités d’affichage et de recherche des restaurants.
J’ai appris à concevoir une interface simple et intuitive pour améliorer l’expérience utilisateur.
La principale difficulté était de gérer les filtres de recherche de manière efficace.
Ce travail m’a permis de mieux comprendre le rôle de l’ergonomie dans une application web.

Elyes Jridi — Développement partie Client

J’ai été responsable de la partie client de l’application.
J’ai développé les fonctionnalités permettant aux utilisateurs de rechercher des restaurants, passer des commandes et laisser des avis.
J’ai appris à gérer les interactions entre plusieurs classes et à structurer la logique métier.
Une difficulté importante était de faire fonctionner correctement les échanges entre les différentes fonctionnalités.
Ce projet m’a permis d’améliorer mes compétences en Java et en conception de systèmes complets.